how to make an llm use a ur5 ?

If you mean **using an LLM as the high-level brain that can command a real UR5**, the practical architecture is:

```
Human
  ↓ natural language
LLM / VLM
  ↓ structured tool call
Robot API / ROS 2
  ↓
Motion planner (MoveIt 2)
  ↓
UR5 controller
  ↓
UR5
```

The important point is: **don't let the LLM directly generate joint commands.** Let it select high-level robot actions, while deterministic robotics software handles kinematics, collision checking, limits, and execution.

1. Give the LLM "tools"

For example, expose functions like:

Python Run

```python
move_to_pose(x, y, z, roll, pitch, yaw)
move_to_named_pose("home")
open_gripper()
close_gripper()
get_robot_pose()
detect_objects()
pick_object("red_cube")
place_object("red_cube", "bin_A")
```

The LLM receives descriptions of these tools and can produce something like:

JSON

```json
{
  "tool": "move_to_pose",
  "arguments": {
    "x": 0.42,
    "y": -0.18,
    "z": 0.31,
    "roll": 3.14,
    "pitch": 0,
```

```
    "yaw": 1.57
  }
}
```

Your program then executes that tool.

---

2. ROS 2 + MoveIt 2 handles the actual motion

For a UR5, I'd use:

```
LLM
 │
 │ tool call
 ↓
Python / ROS 2 node
 │
 ↓
MoveIt 2
 │
 ├── IK
 ├── collision checking
 ├── trajectory generation
 └── joint-limit checking
 │
 ↓
Universal Robots ROS 2 driver
 │
 ↓
UR5
```

So if the LLM says:

> "Move the robot to the cube."

your system should **not** translate that directly into:

```
q1 = ...
q2 = ...
...
```

Instead:

Python Run

```python
pose = perception.get_object_pose("cube")

move_group.set_pose_target(pose)
plan = move_group.plan()
```

```
if plan_successful(plan):
    move_group.execute(plan)
```

MoveIt determines the valid trajectory.

---

3. The LLM should reason at the task level

For example:

User:

> Pick up the blue object and put it in the box.

LLM:

```
1. Find blue object
2. Get object pose
3. Move above object
4. Move down
5. Close gripper
6. Lift
7. Move above box
8. Move down
9. Open gripper
10. Return home
```

The LLM can convert that into tool calls:

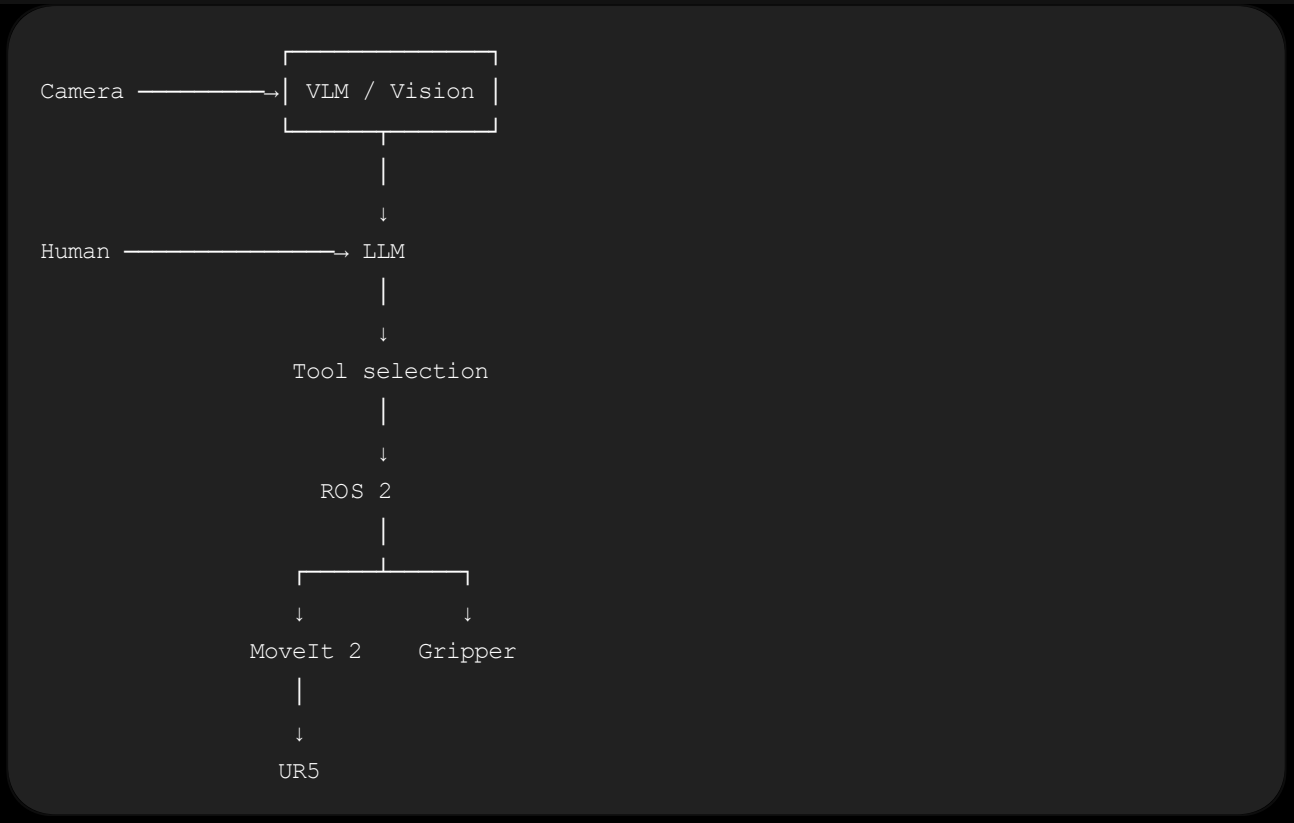
```
detect_objects()
        ↓
get_object_pose("blue_object")
        ↓
move_to_pose(...)
        ↓
close_gripper()
        ↓
move_to_pose(...)
        ↓
open_gripper()
```

This is essentially **LLM-based task planning + conventional robot control**.

---

4. If you want vision too

Then the architecture becomes:



The VLM can answer things such as:

```
What objects are on the table?
Where is the red cube?
Is the gripper aligned with the object?
```

The LLM then uses those observations to decide what to do.

---

5. A very practical implementation

Since you're already familiar with **ROS 2**, I'd build it as:

```
Ubuntu 22.04
ROS 2 Humble
        │
        ├── UR5 ROS 2 driver
        │
        ├── MoveIt 2
        │
        ├── camera node
        │
        ├── perception node
        │
        └── LLM agent
```

The LLM agent could be a Python node:

Python Run

```python
class RobotAgent:

    def move_to_pose(self, pose):
        # call MoveIt

    def open_gripper(self):
        # ROS service/action

    def close_gripper(self):
        # ROS service/action

    def detect_objects(self):
        # call perception

    def get_robot_state(self):
        # TF / joint states
```

Then expose these methods to the LLM as **tools**.

---

6. You can even give the LLM a robot state

For example:

JSON

```json
{
  "robot": "UR5",
  "joint_state": [
    0.12,
    -1.43,
    1.51,
    -1.65,
    -1.57,
    0.02
  ],
  "gripper": "open",
  "objects": {
    "red_cube": [0.42, -0.18, 0.03],
    "blue_cube": [0.31, 0.21, 0.03]
  }
}
```

Then the LLM can reason:

> The blue cube is at x=0.31, y=0.21. Pick it and place it in bin B.

But **the LLM should not be responsible for verifying whether the resulting trajectory is safe**.

---

7. Where an LLM is actually useful

There are roughly three levels:

Level 1 — LLM → predefined robot skills

```
"Pick the red cube"
        ↓
pick("red_cube")
```

Very reliable.

Level 2 — LLM → sequences of skills

```
"Take the cube, put it in the box,
then return to home."

        ↓

pick(cube)
place(cube, box)
move(home)
```

This is probably the **sweet spot** for an academic project.

Level 3 — LLM → low-level control

```
LLM
 ↓
joint velocities
 ↓
UR5
```

I would **not** recommend this architecture. LLM latency, nondeterminism, tokenization, and lack of hard safety guarantees make it inappropriate for direct servoing.

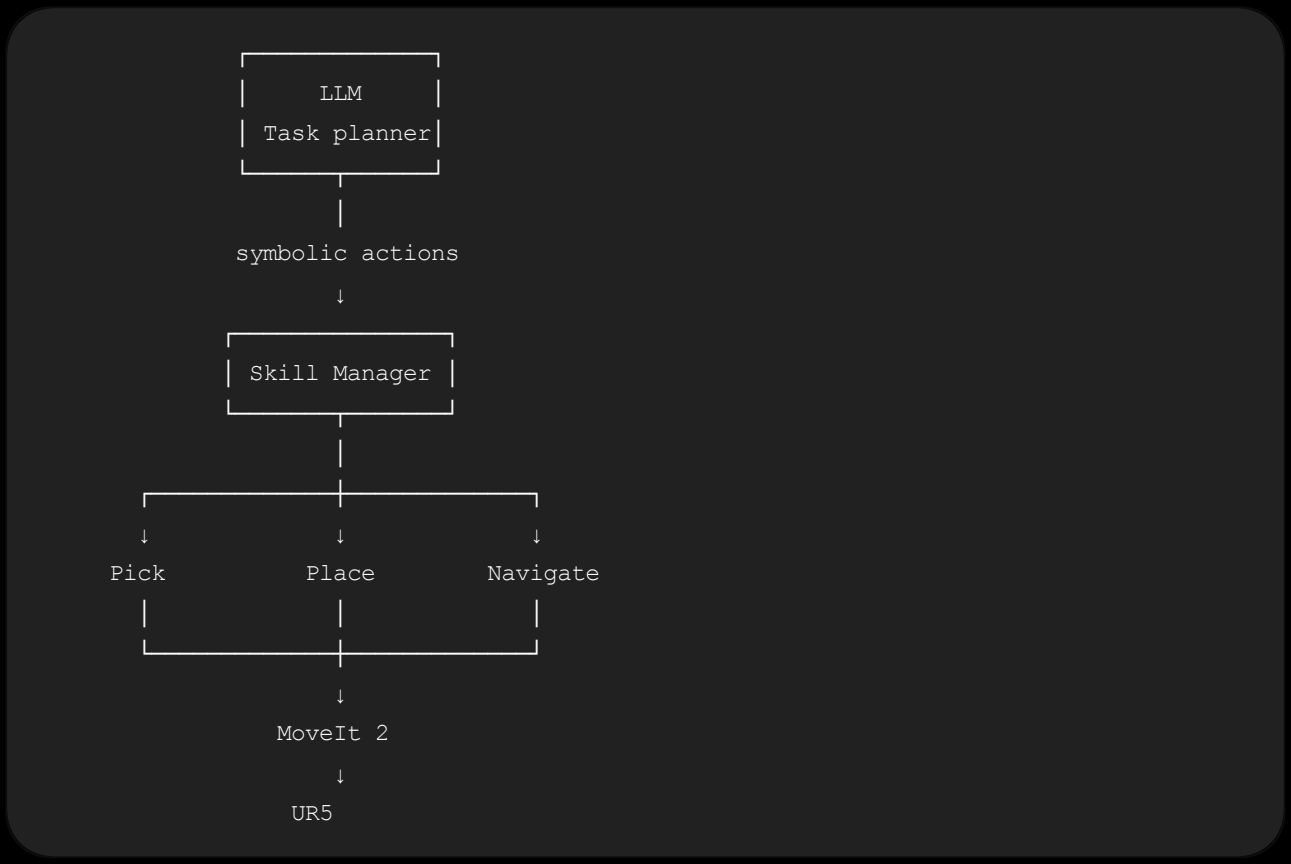
For low-level control, use:

```
MPC / PID / impedance control / operational-space control
```

and let the LLM operate above it.

8. For a research project

Given your robotics background, a more interesting architecture would be:



Then add a **VLM**:

```
Camera → VLM → scene description
                  ↓
                 LLM
                  ↓
            task planning
```

And add a safety layer:

```
LLM
 ↓
Skill
 ↓
Safety / validation
 ↓
MoveIt 2
 ↓
UR5
```

The safety layer can reject:

- unreachable poses

- joint-limit violations
- collision trajectories
- excessive velocities
- workspace violations
- invalid tool arguments

If you want to build this now

I would start **in simulation first**:

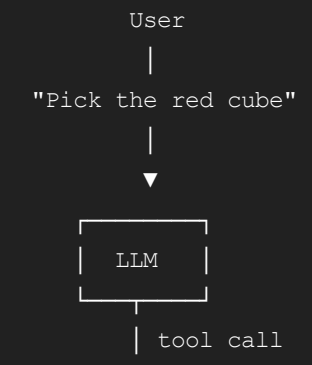
```
UR5
 ↓
Gazebo / Isaac Sim
 ↓
ROS 2
 ↓
MoveIt 2
 ↓
Python LLM agent
```

Once the agent reliably performs tasks in simulation, switch the same ROS interfaces to the physical UR5.

If your goal is specifically **"I want to type `pick up the red object and put it in the box` and have a real UR5 execute it"**, I can give you the complete **ROS 2 + MoveIt 2 + LLM architecture and code**, including the tool-calling Python node and UR5 interface.

using mujoco

Yes. With **MuJoCo**, you can make the LLM control a simulated UR5 without ROS/MoveIt. The clean architecture is:





The key design

Don't have the LLM output:

```
q1 = 0.52
q2 = -1.21
...
```

Instead give it **high-level tools**:

Python Run

```
move_to(x, y, z)
move_to_pose(x, y, z, rx, ry, rz)
open_gripper()
close_gripper()
get_robot_state()
get_object_pose(name)
```

Then your deterministic controller converts the requested pose into joint motion.

For example:

```
LLM:
    pick("red_cube")

Python:
    cube_pose = get_object_pose("red_cube")

    pregrasp = ...
    move_to_pose(pregrasp)

    move_to_pose(cube_pose)
```

```
    close_gripper()

    move_to_pose(lift_pose)
```

MuJoCo handles the physics.

For a UR5 specifically

I'd use:

```
LLM
 │
 │ "pick red cube"
 ▼
Task planner
 │
 ├── get_object_pose()
 ├── move_to_pose()
 ├── close_gripper()
 └── move_to_pose()
 │
 ▼
IK / Cartesian controller
 │
 ▼
MuJoCo
 │
 ▼
UR5
```

For the controller you have two reasonable options.

Option A — MuJoCo IK

Use MuJoCo's Jacobians:

$$\dot{x} = J(q)\dot{q}$$

and calculate joint velocities:

$$\dot{q} = J^{\dagger}(K(x_d - x))$$

Then:

Python Run

```python
J = np.zeros((3, model.nv))
mj_jacSite(model, data, J, None, ee_site)
```

```
error = target_pos - data.site_xpos[ee_site]

dq = np.linalg.pinv(J) @ (K * error)

data.ctrl[:] = dq
```

For a full 6-DOF pose, use the rotational Jacobian too.

Option B — MuJoCo's built-in inverse dynamics / actuator control

For a more realistic setup:

```
desired joint position
        ↓
joint-space PD
        ↓
MuJoCo actuators
        ↓
UR5 dynamics
```

This is easier to stabilize initially.

What makes this interesting for your LLM/VLM work

You can make the LLM operate entirely through a **robot API**.

For example:

Python Run

```python
TOOLS = [
    {
        "name": "get_object_pose",
        "description": "Get the 3D pose of an object in the MuJoCo world."
    },
    {
        "name": "move_end_effector",
        "description": "Move the UR5 end effector to a target pose."
    },
    {
        "name": "open_gripper",
        "description": "Open the gripper."
    },
    {
        "name": "close_gripper",
        "description": "Close the gripper."
    }
]
```

The LLM receives:

> Put the blue cube inside the red container.

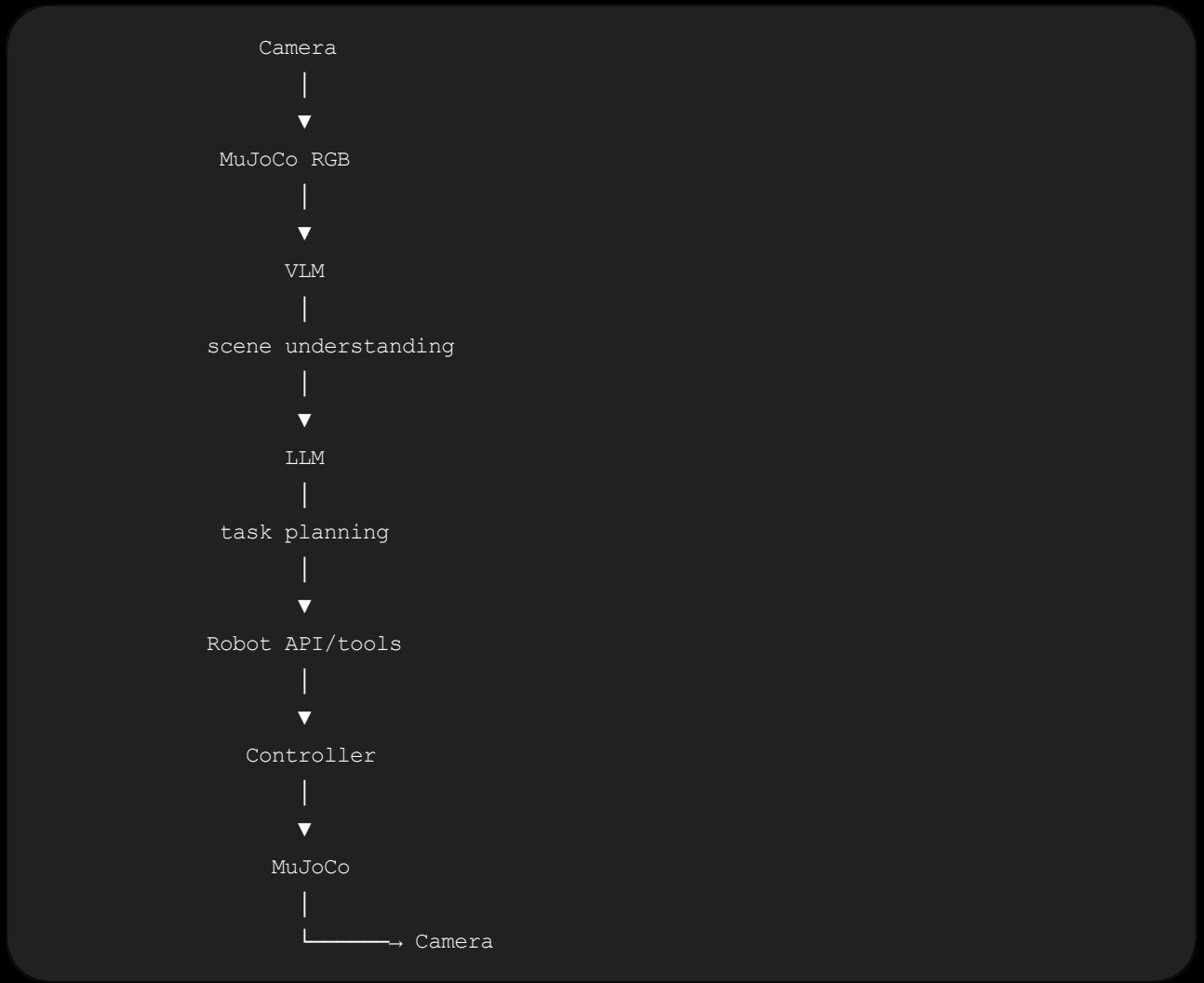
It can generate:

```
get_object_pose("blue_cube")
get_object_pose("red_container")
move_end_effector(...)
close_gripper()
move_end_effector(...)
open_gripper()
```

Your Python code executes each action in MuJoCo and returns the result to the LLM.

You can also give the LLM visual feedback

This becomes a much more interesting **VLM + MuJoCo** setup:



For example:

> "Pick whichever object is closest to the green container."

The VLM identifies the objects, while the LLM decides the sequence of actions.

I would build it in this order

Stage 1

```
MuJoCo UR5
     ↓
Python
     ↓
Cartesian controller
```

Get:

```
move_to_pose()
```

working reliably.

Stage 2

Add:

```
get_object_pose()
open_gripper()
close_gripper()
pick()
place()
```

Stage 3

Connect the LLM as a tool-calling agent.

Stage 4

Add MuJoCo camera → VLM.

Stage 5

Add safety/trajectory validation between the LLM and MuJoCo.

That gives you a proper **LLM/VLM → task planning → robot skills → MuJoCo control** system rather than just asking an LLM to generate robot joint angles.

give implementation guide for levels of complexity. start with very easy . for llm with ur5e

Yes. For **LLM + UR5e + MuJoCo**, I would build it incrementally. Don't start with VLMs, IK, grasping, or autonomous planning. First prove that an LLM can safely invoke a single robot action.

Overall progression

```
Level 0  MuJoCo UR5e
   ↓
Level 1  LLM → simple robot commands
   ↓
Level 2  LLM → multiple predefined motions
   ↓
Level 3  LLM → parameterized Cartesian motions
   ↓
Level 4  LLM → sequential task planning
   ↓
Level 5  LLM → objects + pick/place
   ↓
Level 6  VLM + LLM → visual manipulation
   ↓
Level 7  LLM → autonomous robot agent
```

For your first implementation, **Level 1 should take very little code**.

Level 0 — Get UR5e working in MuJoCo

Goal

Just control the UR5e from Python.

You want:

```
Python
  ↓
MuJoCo
  ↓
UR5e
```

Get a MuJoCo UR5e model with:

- 6 revolute joints

- actuators
- end-effector site
- optionally a gripper

A useful directory structure:

```
llm_ur5e/
├── models/
│   └── ur5e.xml
├── robot.py
├── controller.py
└── main.py
```

Install:

Bash

```
pip install mujoco numpy
```

Test:

Python Run

```python
import mujoco
import mujoco.viewer

model = mujoco.MjModel.from_xml_path("models/ur5e.xml")
data = mujoco.MjData(model)

with mujoco.viewer.launch_passive(model, data) as viewer:
    while viewer.is_running():
        mujoco.mj_step(model, data)
        viewer.sync()
```

At this stage, **no LLM**.

Level 1 — LLM → predefined robot commands

This is where I recommend you start.

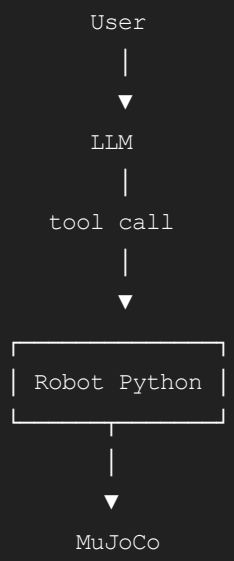
Don't give the LLM continuous control.

Give it only:

```
home()
wave()
```

```
move_left()
move_right()
```

Architecture:



For example:

> Move the UR5e to the home position.

LLM produces:

JSON

```
{
    "action": "home"
}
```

Your Python program executes:

Python Run

```
def home():
    target_q = np.array([
        0,
        -1.57,
        0,
        -1.57,
        0,
        0
    ])

    controller.move_joints(target_q)
```

Important

The LLM **doesn't control MuJoCo directly**.

It only chooses:

```
home
```

Your code determines what `home` actually means.

Level 2 — LLM chooses from predefined poses

Now give the LLM:

```
home()
ready()
left()
right()
front()
```

For example:

> Put the robot in the ready position.

LLM:

JSON

```
{
    "action": "move_to_named_pose",
    "pose": "ready"
}
```

Python:

Python Run

```
NAMED_POSES = {
    "home": np.array([...]),
    "ready": np.array([...]),
    "left": np.array([...]),
    "right": np.array([...])
}
```

Then:

Python Run

```
def move_to_named_pose(name):

    if name not in NAMED_POSES:
        raise ValueError("Invalid pose")

    controller.move_joints(NAMED_POSES[name])
```

Why this level matters

You're testing:

natural language → structured action → robot

without introducing difficult robotics problems.

Level 3 — LLM → Cartesian position

Now things become more interesting.

Instead of:

```
move_to("left")
```

give the LLM:

Python Run

```
move_to_position(x, y, z)
```

User:

> Move the end effector to x=0.4, y=-0.2, z=0.3.

LLM:

JSON

```
{
    "action": "move_to_position",
    "x": 0.4,
    "y": -0.2,
    "z": 0.3
}
```

Your controller does:

```
Cartesian target
       ↓
      IK
       ↓
joint target
       ↓
   controller
       ↓
     MuJoCo
```

This is where you introduce your own IK.

For UR5e:

$$x_d = \begin{bmatrix} x_d & y_d & z_d \end{bmatrix}^T$$

Calculate:

$$e = x_d - x(q)$$

and use:

$$\dot{q} = J^{\dagger} K e$$

Then integrate:

$$q_{t+1} = q_t + \dot{q}\Delta t$$

This gives you a basic Cartesian controller.

Level 4 — LLM as a task planner

Now expose several skills:

```
move_to_position()
open_gripper()
close_gripper()
move_home()
```

The user can say:

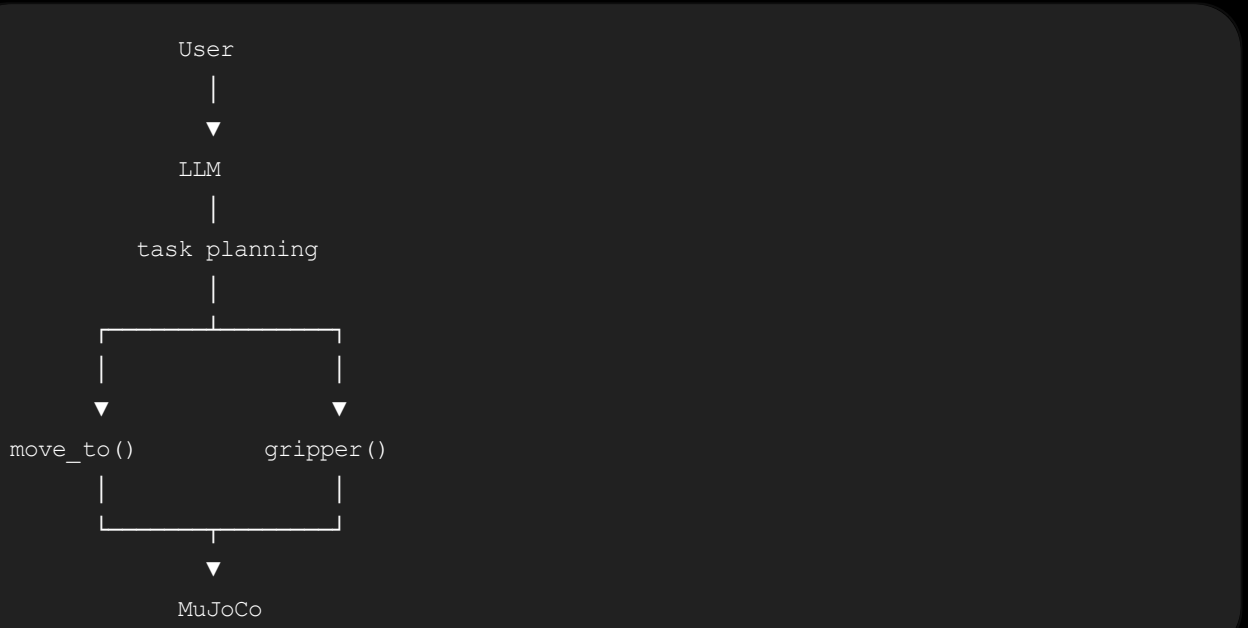
> Move forward, lower the arm, close the gripper, then return home.

LLM decomposes it:

```
1. move_to_position(...)
2. move_to_position(...)
```

```
3. close_gripper()
4. move_home()
```

Architecture:



Now you have a genuine **LLM task planner**.

Level 5 — Pick and place

Now add objects to MuJoCo.

Scene:



Expose:

Python Run

```python
get_object_pose(name)
pick(name)
place(name, location)
```

The LLM sees:

```
red_cube:
    position = [0.42, -0.15, 0.04]

box:
    position = [0.55, 0.20, 0.04]
```

User:

> Put the red cube in the box.

LLM:

```
get_object_pose("red_cube")

pick("red_cube")

place("red_cube", "box")
```

But `pick()` itself should be deterministic:

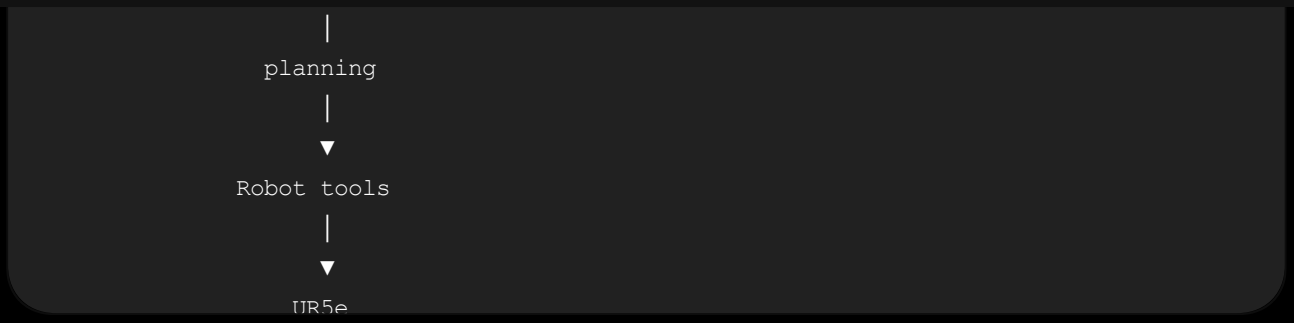
```
pick()
 │
 ├── move above object
 ├── descend
 ├── close gripper
 └── lift
```

So the LLM doesn't need to figure out every millisecond of motion.

Level 6 — Add VLM

Now introduce camera input.

```
                MuJoCo
                  │
                Camera
                  │
                  ▼
                VLM
                  │
          scene description
                  │
                  ▼
                LLM
```



Example:

> Pick up the red object and put it in the container.

VLM:

```
Objects:
- red cube at (0.41,-0.12)
- blue cube at (0.30,0.15)
- container at (0.55,0.2)
```

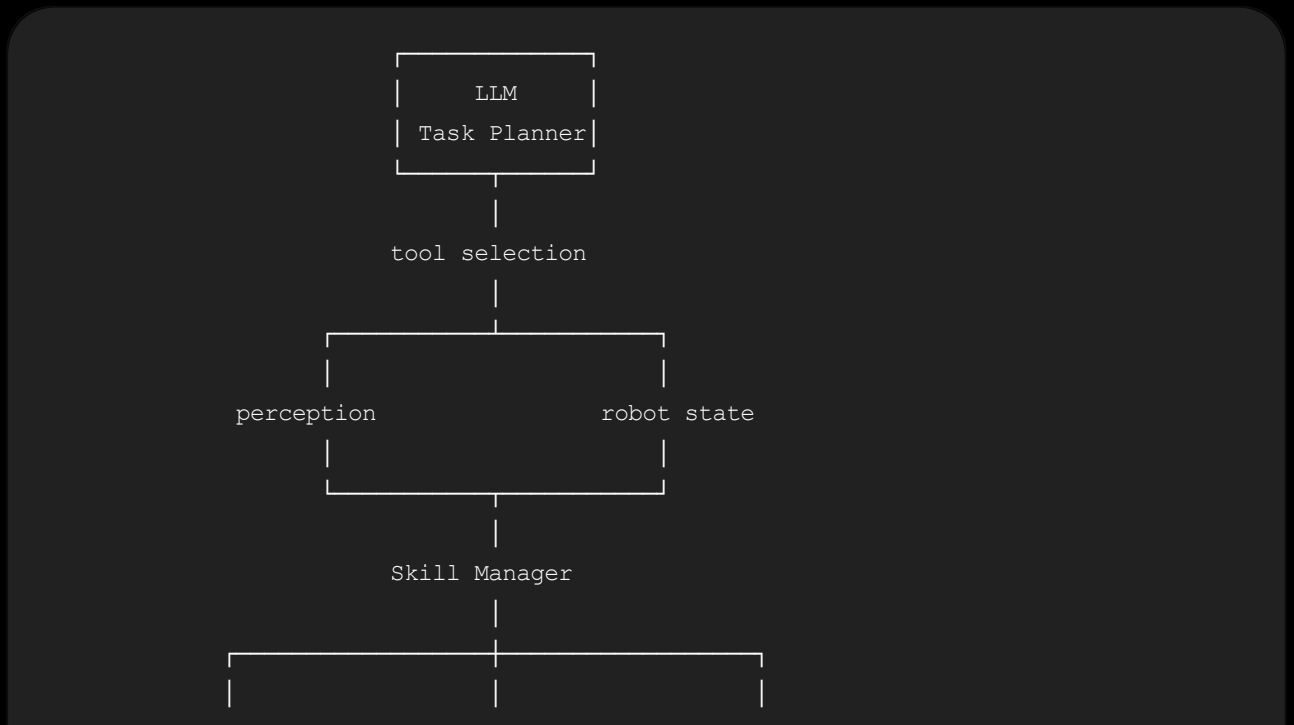
LLM:

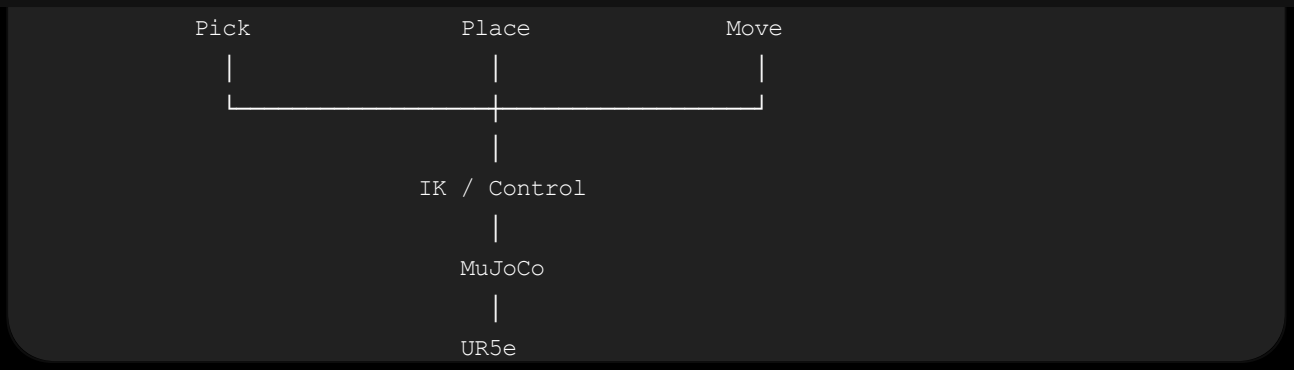
```
pick(red_cube)
place(red_cube, container)
```

This is substantially more interesting as a robotics project.

Level 7 — Full LLM robot agent

Eventually:





And importantly:

```
LLM
 ↓
high-level actions
 ↓
validation
 ↓
robot controller
 ↓
MuJoCo
```

Never:

```
LLM → arbitrary joint torque
```

What I recommend you implement first

For your first working prototype, keep it extremely small:

```
Level 1
```

Files

```
llm_ur5e/
├── models/
│   └── ur5e.xml
├── robot.py
├── llm.py
└── main.py
```

Robot API

Only:

Python Run

```python
move_home()
move_ready()
move_left()
move_right()
```

LLM API

Give the LLM these tools:

```
move_home
move_ready
move_left
move_right
```

Then test:

```
User:
"Move the robot to the left."

LLM:
move_left()

MuJoCo:
UR5e moves left.
```

Then:

```
User:
"Put the robot in the ready position."

LLM:
move_ready()

MuJoCo:
UR5e moves to ready.
```

Once this works reliably, **Level 2 is almost trivial**.

Recommended progression for your project

Level	LLM capability	Robotics difficulty
0	No LLM	★

Level	LLM capability	Robotics difficulty
1	Select predefined action	★
2	Select named pose	★
3	Cartesian target	★★
4	Multi-step planning	★★
5	Pick & place	★★★
6	VLM + manipulation	★★★★
7	Autonomous agent	★★★★★

I'd specifically target **Levels 3 → 5** first. That gives you a meaningful **LLM-controlled UR5e in MuJoCo** while keeping the low-level control deterministic and reproducible.

If you already have a **UR5e MuJoCo XML/model**, the next concrete step is to build **Level 1 from scratch with the complete `main.py`, `robot.py`, and LLM tool-calling code**.

how can jev / maya be used here .

Yes. **Jev fits your MuJoCo UR5e setup particularly well as the decision layer**, rather than as the LLM that writes a whole plan.

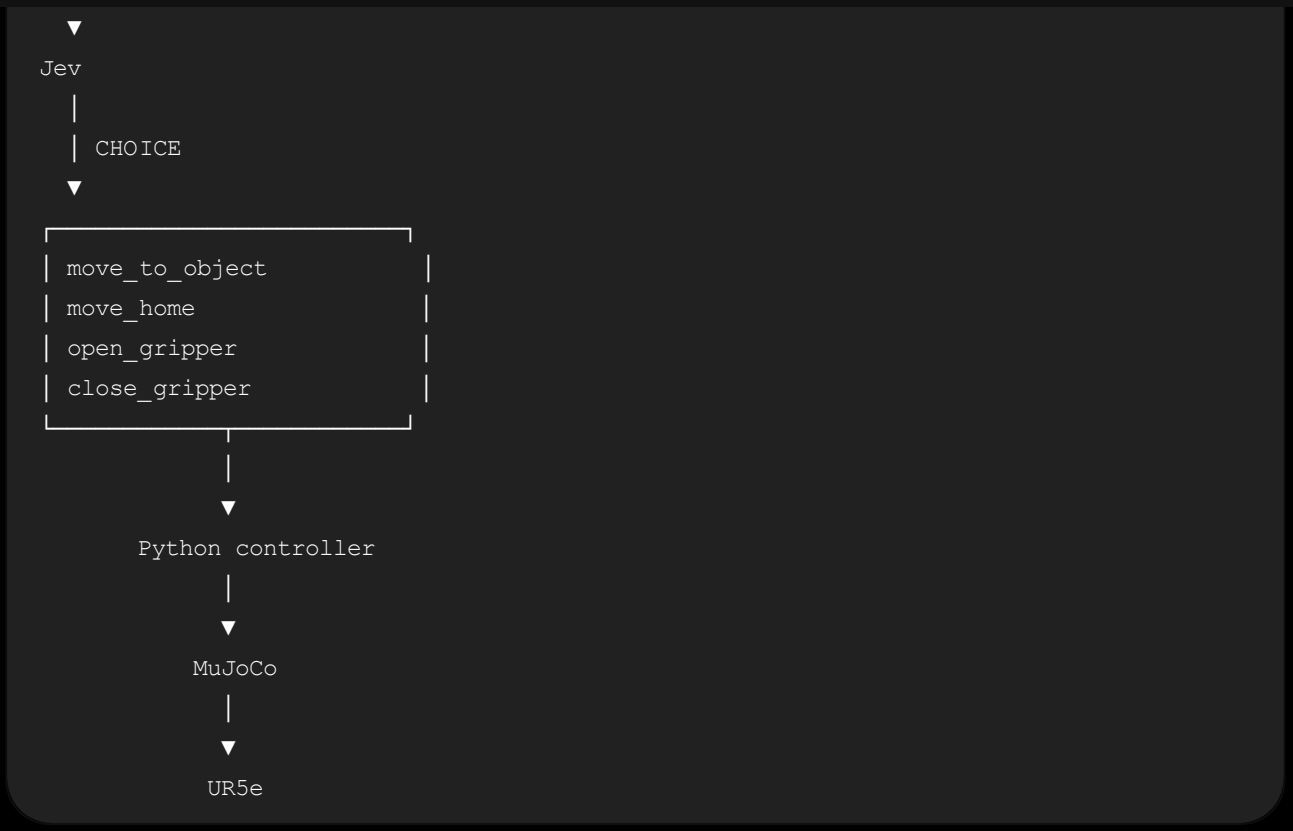
There are already recent MuJoCo demonstrations showing essentially this architecture: Jev receives structured simulator state, chooses a discrete action, and a conventional controller executes the motion. One published xArm7 example uses Jev to choose intent plus X/Y/Z movement directions and gripper state; the executor handles IK and incremental physics. GitHub

For your **UR5e**, I would build it like this.

1. Very easy: Jev chooses between robot skills

Start with:

```
User
  |
  | "move the robot to the object"
```



Jev is designed around **typed decisions** rather than generating a text response, so this is a natural fit. It can select one option from a predefined set and return probabilities for the choices. GitHub

For example, send:

JSON

```
{
  "state": {
    "ee_distance_to_object": 0.18,
    "gripper": "open",
    "object_grasped": false,
    "target_reached": false
  },
  "question": "What should the robot do next?"
}
```

Possible choices:

```
[
    "move_toward_object",
    "open_gripper",
    "close_gripper",
    "move_home",
    "finish"
]
```

Jev returns something like:

```
move_toward_object
```

Then **your code** executes it.

---

2. Slightly harder: Jev controls incremental movement

This is the architecture I'd recommend for your first actual experiment.

Instead of giving Jev:

```
move_to_pose(x,y,z)
```

give it discrete choices:

```
X: left / hold / right
Y: forward / hold / backward
Z: up / hold / down

Gripper:
open / hold / close
```

So one Jev decision could be:

JSON

```
{
    "x": "positive",
    "y": "negative",
    "z": "hold",
    "gripper": "open"
}
```

Your controller translates that into:

Python Run

```
target_xyz += direction * step_size
```

Then:

```
target XYZ
    ↓
IK
    ↓
UR5e joint target
```

```
    ↓
MuJoCo
```

This is very close to the recently published Jev + MuJoCo xArm7 experiment. Their executor used small 18/4/2 mm increments depending on proximity to the target rather than letting the model produce arbitrary joint torques or trajectories. GitHub

For your UR5e

I'd start with:

```python
STEP = 0.01  # 10 mm
```

and actions:

```python
X_ACTIONS = [
    "negative",
    "hold",
    "positive"
]

Y_ACTIONS = [
    "negative",
    "hold",
    "positive"
]

Z_ACTIONS = [
    "negative",
    "hold",
    "positive"
]
```

Then Jev becomes something like a **high-level reactive controller**.

---

3. Add an explicit state machine

Now don't ask Jev:

> What should the robot do?

Instead give it the current **task state**.

For example:

```
APPROACH
GRASP
LIFT
MOVE
PLACE
RELEASE
FINISH
```

Jev chooses:

```
APPROACH
```

then:

```
MOVE_X_POSITIVE
```

then:

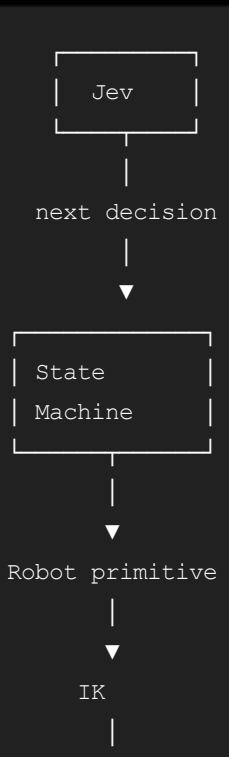
```
MOVE_Z_NEGATIVE
```

then:

```
GRASP
```

etc.

Architecture:



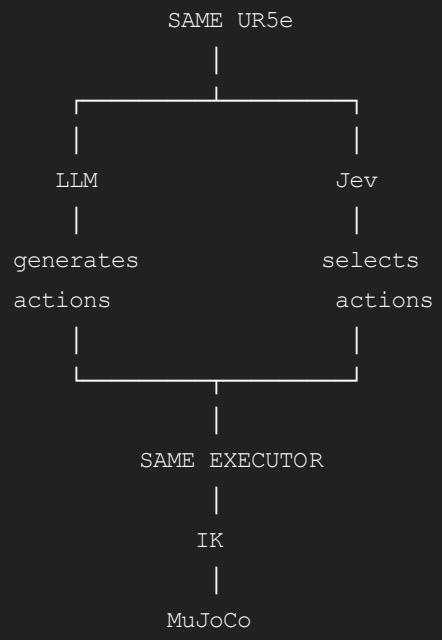
▼

This is safer because **Jev isn't responsible for remembering the entire manipulation procedure**.

---

4. This is where Jev vs LLM becomes interesting

You can actually compare:



For example:

GPT/LLM

Input:

```
The object is 8 cm to the right,
3 cm forward and 5 cm below the TCP.
What should I do?
```

Output:

```
Move right and slightly forward...
```

You then have to parse/validate that.

Jev

Define:

```
X ∈ {negative, hold, positive}
Y ∈ {negative, hold, positive}
Z ∈ {negative, hold, positive}
```

and let Jev directly choose from those typed options.

That's one of Jev's main advantages for this kind of application: **your action space is explicitly defined by your robotics code rather than inferred from generated text.** GitHub

---

5. Add confidence-based safety

This is particularly interesting.

Suppose Jev returns:

```
move_positive_x
confidence = 0.91
```

Your controller can have:

Python Run

```python
if confidence > 0.85:
    execute()
else:
    request_new_observation()
```

Or:

```
confidence > 0.90
        ↓
execute

0.60–0.90
        ↓
smaller movement

< 0.60
        ↓
stop / ask another model
```

The Jev documentation explicitly describes confidence as useful for routing decisions, including automatically acting at high confidence and routing low-confidence cases to another system or a human. GitHub

Important: don't interpret the probability as "91% probability the robot action will succeed." The model's output probability is not automatically a calibrated physical-success probability. The robotics system needs its own validation/calibration. GitHub

---

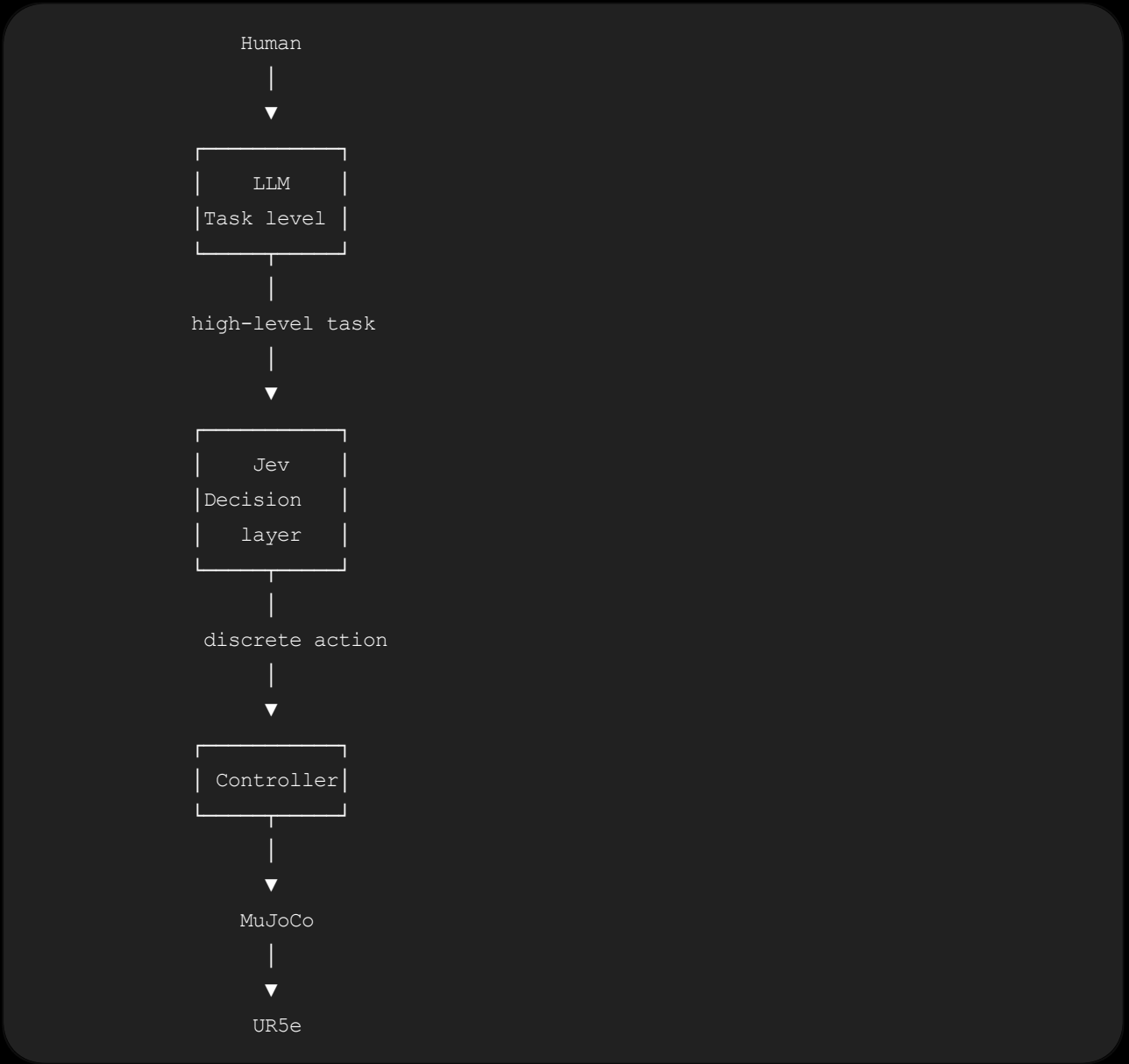
6. Add an LLM above Jev

This is where I think your project gets much more interesting.

Instead of:

```
LLM → UR5e
```

use:



Example:

Human

> Pick the red cube and put it in the box.

LLM

Creates the task:

```
1. Approach cube
2. Align gripper
3. Grasp
4. Lift
5. Move to box
6. Release
```

Jev

At each step decides:

```
What should I do next?
```

based on:

JSON


```
{
    "object_distance": 0.063,
    "object_grasped": false,
    "ee_height": 0.31,
    "target_distance": 0.42,
    "collision": false
}
```


and selects:

```
MOVE_RIGHT
```

Then MuJoCo executes it.

---

7. Maya can fit at the top of this

If by **Maya** you mean a Maya/animation-oriented agent/workflow, I would use it for **demonstration/scene/task authoring**, not as the low-level UR5 controller.

For example:

```
Maya
 |
 | scene/task specification
 ▼
UR5e task representation
 |
 ▼
LLM
 |
 ▼
Jev
 |
 ▼
MuJoCo
```

You could create:

```
Maya scene
    ↓
object positions
target locations
robot/environment configuration
    ↓
export structured scene
    ↓
MuJoCo
```

Then the LLM/Jev operates on that environment.

If by **Maya** you mean a specific AI/robotics framework called Maya, tell me which one/link you mean—the name is ambiguous, and I don't want to assume the wrong system.

---

8. The experiment I'd actually recommend

You can make this a clean research experiment:

Environment

```
MuJoCo
UR5e
table
cube
target
gripper
```

Controllers

```
A. Scripted controller
B. GPT/LLM
C. Jev
D. LLM → Jev
```

All four use the **same**:

```
state
 ↓
action space
 ↓
IK
 ↓
controller
 ↓
MuJoCo
```

Then measure:

```
Success rate
Number of decisions
Number of environment steps
Time
API latency
API cost
Path length
Number of collisions
Number of failed grasps
```

This is much cleaner scientifically than comparing completely different robot architectures.

There is already precedent for this style of evaluation: the open MuJoCo Jev experiment compares Jev and GPT controllers using the same physical executor and reports decision cycles, API calls, cost, wall time and simulation time. GitHub

And there are now other MuJoCo studies using Jev as a **task-level decision maker while keeping the robot skill/controller and safety veto in code**, which is almost exactly the architecture I'd use for your UR5e. GitHub

My suggested progression

```
                UR5e + MuJoCo
                     |
                Level 1
                     |
          Jev → {home, ready, left}
                     |
```

```
                ▼
           Level 2
                |
Jev → X/Y/Z incremental movement
                |
                ▼
           Level 3
                |
    Jev → pick/place skills
                |
                ▼
           Level 4
                |
  LLM → task decomposition
                |
                ▼
           Level 5
                |
     LLM → Jev → UR5e
                |
                ▼
           Level 6
                |
      VLM → LLM → Jev
```

For your first implementation, I would skip Maya entirely and build Level 2: `Jev → discrete Cartesian action → IK → MuJoCo UR5e`**.** That is small enough to get running quickly but already demonstrates the interesting part of Jev for robotics.

ChatGPT can make mistakes. Check important info.